Bitácora de comandos — Proyecto Traductor LSM

Registro completo y cronológico de todos los comandos usados, para qué sirvió cada uno, y en qué momento del proceso se usó.

FASE 1: Preparación del entorno

Verificar que Python está instalado

python --version

Confirma que Python funciona y muestra la versión (en tu caso, 3.13).

Moverse a la carpeta del proyecto

cd C:\ProyectoLSM

Le dice a la terminal "trabaja dentro de esta carpeta". Se usa CADA VEZ que abres una terminal nueva, antes de correr cualquier otro comando del proyecto.

Instalar las librerías necesarias

pip install -r requirements.txt

Lee el archivo requirements.txt e instala automáticamente todas las librerías que lista (tensorflow, mediapipe, opencv-python, pandas, numpy, matplotlib, seaborn, scikit-learn, tqdm).

FASE 2: Organizar el dataset

Ver el contenido de una carpeta (usado varias veces para explorar el dataset)

dir MSL-ABC

dir MSL-ABC\MSL-ABC

dir lista lo que hay dentro de una carpeta. Se usó para descubrir que el dataset tenía una carpeta duplicada por dentro (MSL-ABC\MSL-ABC\lsm-abc-A, etc.) antes de saber la ruta correcta para el siguiente paso.

Juntar los 3 grupos del dataset en una sola estructura

python 00_merge_groups.py --input MSL-ABC\MSL-ABC --output dataset

Corre el script que junta lsm-abc-A, lsm-abc-B, lsm-abc-C (cada uno con su propio train/test) en una sola carpeta dataset/train/LETRA y dataset/test/LETRA.

Nota: la primera vez este comando falló con error de espacio en disco (OSError: [WinError 112]) porque el script copiaba los archivos (duplicando el espacio necesario). Se corrigió el script para que MOVIERA los archivos en vez de copiarlos, y se liberó espacio en disco antes de reintentar.

Borrar una carpeta con todo su contenido (usado para limpiar el intento fallido)

rmdir /s dataset

Borra la carpeta dataset y todo lo que tenga adentro. Pide confirmación (escribe S y Enter). Se usó para eliminar el resultado parcial del primer intento que falló por falta de espacio, antes de volver a correr 00_merge_groups.py con la versión corregida del script.

FASE 3: Extraer landmarks de las imágenes

Correr el script de extracción de landmarks

python 05_extract_landmarks.py --data_dir dataset --output_dir landmarks_data

Recorre dataset/train y dataset/test, detecta la mano en cada imagen con MediaPipe, extrae y normaliza los 21 landmarks, y guarda todo en landmarks_data/landmarks_train.csv y landmarks_data/landmarks_test.csv. Por default toma una muestra de 600 imágenes de train por letra (120 en test) en vez de procesar el dataset completo (~245,000 imágenes), para que sea viable en el tiempo disponible.

Nota: este comando falló varias veces antes de funcionar, con el error AttributeError: module 'mediapipe' has no attribute 'solutions', porque tu versión de mediapipe (1.0.1) usa una API distinta a la que tenía el script originalmente. Hubo que:

1. Descargar el modelo de MediaPipe por separado (ver siguiente sección)
2. Reescribir el script para usar la API nueva (HandLandmarker en vez de mp.solutions.hands)

Comandos usados para diagnosticar el problema de la versión vieja del script

findstr "HandLandmarker" 05_extract_landmarks.py

Busca la palabra "HandLandmarker" dentro del archivo, mostrando las líneas donde aparece. Se usó para confirmar si el archivo en la carpeta del proyecto realmente tenía la versión actualizada del código, o si seguía siendo la versión vieja (que no contiene esa

palabra). Cuando no mostraba nada, confirmaba que había que volver a descargar el archivo.

dir C:\Users\Surface\Downloads\05_extract_landmarks*

Verifica si el archivo descargado desde el chat cayó en la carpeta de Descargas, y con qué nombre exacto (a veces Windows le agrega "(1)" si ya existía otro archivo con ese nombre).

del 05_extract_landmarks.py

Borra el archivo (versión vieja) para evitar confusión antes de reemplazarlo.

move C:\Users\Surface\Downloads\05_extract_landmarks.py C:\ProyectoLSM\05_extract_landmarks.py

Mueve el archivo recién descargado desde la carpeta de Descargas hasta la carpeta del proyecto, reemplazando la ubicación.

Descarga del modelo de MediaPipe (no es un comando de terminal, se hizo desde el navegador)

Se descargó manualmente el archivo hand_landmarker.task desde:

https://storage.googleapis.com/mediapipe-models/hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task

y se movió a C:\ProyectoLSM\hand_landmarker.task. Este archivo contiene el modelo de detección de manos ya entrenado por Google, necesario para que MediaPipe funcione con la API nueva.

FASE 4: Entrenar el modelo

Correr el entrenamiento

python 06_train_landmarks_model.py --data_dir landmarks_data --output_dir model_output

Carga los CSV de landmarks, entrena una red neuronal densa (no una CNN de imágenes) sobre esos datos, guarda el modelo entrenado, gráficas de accuracy/loss, y un diagnóstico de si el modelo aprendió o memorizó.

Resultado obtenido: accuracy train 91.8%, accuracy validación 92.5% — diagnóstico de buena generalización, sin sobreajuste.

Verificar los archivos generados

dir model_output

Confirma que se crearon correctamente: class_indices.json, lsm_landmarks_model_final.h5, training_graphs.png, training_history.csv.

Abrir la gráfica de entrenamiento generada

start model_output\training_graphs.png

start abre un archivo con el programa por default de Windows (en este caso, el visor de imágenes), directo desde la terminal.

FASE 5: Evaluar el modelo

Correr la evaluación completa

python 07_evaluate_landmarks.py --mode evaluate --data_dir landmarks_data --model model_output/lsm_landmarks_model_final.h5

Evalúa el modelo entrenado contra el set de test completo (2,489 imágenes). Genera un reporte de precisión por letra y una matriz de confusión guardada como imagen en model_output/confusion_matrix.png.

Resultado obtenido: 93% de accuracy general. Confusiones principales entre R/U/V y M/N (letras con formas de mano visualmente similares).

FASE 6: Traductor en vivo (resultado final)

Correr el traductor con fondo negro (opción por default)

python 08_realtime_translator.py

Correr el traductor mostrando la webcam real (no fondo negro)

python 08_realtime_translator.py --background video

Abre la webcam, detecta la mano en tiempo real, predice la letra, y permite armar palabras con la barra espaciadora.

Controles dentro de la ventana:

- ESPACIO → confirma la letra actual y la agrega a la palabra
- BACKSPACE → borra la última letra

- C → limpia la palabra completa
- ESC → cierra la ventana y termina el programa

Comandos generales de navegación usados a lo largo de todo el proceso

Comando	Para qué sirve
cd NOMBRE_CARPETA	Entra a una carpeta
cd ..	Sube un nivel (sale de la carpeta actual hacia la carpeta contenedora)
dir	Lista el contenido de la carpeta actual
dir RUTA	Lista el contenido de una carpeta específica sin necesidad de entrar a ella
del NOMBRE_ARCHIVO	Borra un archivo
rmdir /s NOMBRE_CARPETA	Borra una carpeta completa con todo su contenido (pide confirmación)
move ORIGEN DESTINO	Mueve (o renombra) un archivo o carpeta
start ARCHIVO	Abre un archivo con el programa por default de Windows

---

Resumen de la secuencia completa, de principio a fin

cd C:\ProyectoLSM

pip install -r requirements.txt

python 00_merge_groups.py --input MSL-ABC\MSL-ABC --output dataset

python 05_extract_landmarks.py --data_dir dataset --output_dir landmarks_data

python 06_train_landmarks_model.py --data_dir landmarks_data --output_dir model_output

python 07_evaluate_landmarks.py --mode evaluate --data_dir landmarks_data --model model_output/lsm_landmarks_model_final.h5

python 08_realtime_translator.py --background video

Estos 6 comandos (sin contar instalación ni los pasos de depuración de errores) son el corazón del pipeline: de dataset crudo a traductor funcionando en vivo.